Week 6 Level 2: Intermediate

Review and Checkpoint

Bugs are normal: detectives find them with a strategy, not by luck.

What you will learn

- How to read a **traceback** from the bottom up.
- Three more detective tools: **print-debugging**, the **rubber duck** and the **trace table**.
- How to write a short **bug report**, so you (and others) remember what you learned.

The big idea: debug like a detective

Every programmer writes bugs. Good programmers find them with a **strategy**: they collect clues, change **one** thing and test again.

Tool 1: read the traceback from the bottom up

When a program crashes, Thonny shows a red **traceback** like this one:

Output

```
Traceback (most recent call last):
  File "starter.py", line 24, in <module>
  File "starter.py", line 17, in make_card
  File "starter.py", line 10, in card_line
    return label + ": " + value
TypeError: can only concatenate str (not "int") to str
```

1. **WHAT** went wrong: the **last line**. Here: a `TypeError`, text + number.
2. **WHERE**: the `File` line just above it. Here: line 10, in the function `card_line`.
3. **HOW** it got there: the lines further up. The main program (line 24) called `make_card`, which called `card_line`.

In Thonny you can click the blue link to jump straight to that line.

Error name	It usually means…
`TypeError`	two types that do not go together, like text + number
`IndexError`	a list index that does not exist (remember: the last index is `len - 1`)
`KeyError`	a dictionary key that does not exist (check the spelling)
`NameError`	a variable name Python does not know (typo, or missing quotes)
`ValueError`	the right type but a bad value, like `int("banana")`

Tool 2: print-debugging

Not sure what is inside a variable? Ask Python! Add a `print()` line, run, look, then remove it.

```
print("best is", best, "length", len(best))
```

Tool 3: the rubber duck

Explain your code, line by line, out loud, to a partner, a toy or a rubber duck. Say what each line *should* do. Very often you hear yourself say the bug before you reach the end. This is the best tool for bugs that do not crash.

Tool 4: the trace table

Follow the code with a pencil, like a very slow computer. Write a column for each variable and a row each time round the loop. When the table shows a value you did not expect, you have found the bug.

Tip: Bugs are normal

If you are stuck, you are not bad at coding: you are in the middle of learning. Say “I can’t fix this *yet*”, pick a tool and try one thing.

Bug-hunt workshop

Each station has a folder in `exercises/week06`. Open `starter.py`, find the bug, fix it, then check with `check_my_work.py`. Fill in a bug report for each bug you fix.

Exercise 6.1: Station 1: Hero Card (TypeError)

Folder: `ex-bug-hero-card`. We do this one together.

Error name: _____ Line: _____ Tool I used: _____

I expected: _____ It really did: _____

What I changed:

_____

Exercise 6.2: Station 2: High Scores (IndexError, 2 bugs)

Folder: `ex-bug-high-scores`. Draw the list with its index numbers. Try `print("best is", best)` inside `podium()`.

Bug 1 – error name: _____ line: _____ what I changed: _____

Bug 2 – error name: _____ line: _____ what I changed: _____

Exercise 6.3: Station 3: Rocket Countdown (infinite loop)

Folder: `ex-bug-countdown`. The safety net stops the stuck loop. Fill in the trace table for `countdown(5)` as the starter is now:

round	n	numbers
1	5	[5]
2		
3		

What should change each time round, but does not? _____

What I changed:

_____

Exercise 6.4: Station 4: Loot Shop (3 sneaky bugs, no crash)

Folder: `ex-bug-loot-shop`. First work it out on paper for the basket `[12, 5, 30]`:

Total: _____ Cheapest: _____ Can you buy it all with `50` coins? _____

Now run it and explain each function to your duck. The three bugs I fixed:

_____

_____

Exercise 6.5: Station 5: Paper detectives (no computer)

This function should give back the biggest number in a list. Fill in the trace table for `biggest([-5, -2, -8])`.

```
def biggest(numbers):
    best = 0
    for n in numbers:
        if n > best:
            best = n
    return best
```

n	n > best?	best
start	–	0
-5		
-2		
-8		

It gives back _____, but the biggest number is _____. What is the bug, and how would you fix it?

Vocabulary box

bug A mistake in a program that makes it crash or give the wrong answer.

debugging Finding and fixing bugs, step by step, with a strategy.

traceback The red error message. Read it from the bottom up: what, then where.

print-debugging Adding `print()` lines to see what is really inside your variables.

rubber duck debugging Explaining your code line by line out loud, to find the bug yourself.

trace table A table where you follow code by hand and write each variable's value.

bug report A short note: what went wrong, where, and what you changed.

Reflection

Six weeks ago I could not … and now I can …

Which detective tool will you try first next time? Why?

Week 6 Level 2: Intermediate

Review and Checkpoint

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Pixel Pet bug hunt (Thonny, 15 minutes)

Open `exercises/week06/hw-bug-pixel-pet/starter.py`. It is a virtual pet stored in a dictionary, and it has **three** bugs. The rules: `feed()` takes 3 from hunger (never below `0`); `play()` adds 2 to happiness and 1 to hunger; `mood()` says `"grumpy"` if hunger is 8 or more **or** happiness is 2 or less, otherwise `"happy"`.

1. Press **Run**. Read the traceback bottom-up and fix bug 1.
2. Run again and look at the printed dictionary. Is there a key that should not be there? That is bug 2.
3. Bug 3 does not crash. Explain `mood()` out loud (rubber duck!) and compare it with the rules.
4. Open `check_my_work.py` and press **Run**.

Bug	Error name (or “no crash”)	Line	What I changed
1			
2			
3			

★★ Challenge (want more?): Trace it

Fill in a trace table on scrap paper and write what this prints:

```
coins = 1
days = 0
while coins < 20:
    coins = coins * 2
    days = days + 1
print(days, coins)
```

It prints: _____

Creative (make it yours): Bug comic

Draw a 3-panel comic about a bug and the detective who finds it. Panel 1: the program breaks. Panel 2: the detective uses one of the four tools. Panel 3: fixed! Give your bug a name.

Self-check: tick each box when it is true

- ☐ I read tracebacks from the bottom up.
- ☐ I used at least two different detective tools this week.
- ☐ I wrote a bug report for each Pixel Pet bug.
- ☐ When I got stuck, I tried one thing at a time instead of changing everything.